```
# Fashion MNIST Classification using CNN
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras import layers, models #

# Load data from CSV files
train_df = pd.read_csv('fashion-mnist_train.csv')
test_df  = pd.read_csv('fashion-mnist_test.csv') #

# Separate labels (y) from pixel values (X)
y_train = train_df['label'].values         # Class 0-9
X_train = train_df.drop('label', axis=1).values
y_test  = test_df['label'].values
X_test  = test_df.drop('label', axis=1).values #

# Reshape flat 784-pixel rows into 28x28 images
X_train = X_train.reshape(-1, 28, 28, 1)    # -1 = all samples, 1 = grayscale
X_test  = X_test.reshape(-1, 28, 28, 1) #

# Normalize pixel values from 0-255 to 0-1
X_train = X_train / 255.0                # Divide all by 255
X_test  = X_test  / 255.0 #

# Class names for the 10 clothing categories
class_names = ['T-shirt','Trouser','Pullover','Dress',
           'Coat','Sandal','Shirt','Sneaker',
           'Bag','Ankle boot'] #

# Build the CNN model
model = models.Sequential([
  # Conv layer: 32 filters of size 3x3, detects features like edges
  layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
  # MaxPooling: reduces image size by taking max in each 2x2 block
  layers.MaxPooling2D(2, 2),
  # Second conv layer: 64 filters, detects more complex features
  layers.Conv2D(64, (3,3), activation='relu'),
  layers.MaxPooling2D(2, 2),
  # Flatten 2D maps to 1D vector for Dense layers
  layers.Flatten(),
  layers.Dense(128, activation='relu'),      # Fully connected layer
  layers.Dropout(0.3),                    # Prevent overfitting
  # Output: 10 neurons (one per class), softmax gives probabilities
  layers.Dense(10, activation='softmax')
]) #

# Compile model for multi-class classification
model.compile(
  optimizer='adam',
  loss='sparse_categorical_crossentropy',   # For integer labels (not one-hot)
  metrics=['accuracy']
) #

model.summary()                         # Print layers and parameters

# Train the model
model.fit(X_train, y_train,
```

```
        epochs=10,                  # 10 passes through training data
        batch_size=64,
        validation_split=0.1,
        verbose=1) #

model.summary()

# Evaluate on test data
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {test_acc*100:.2f}%") #

# Predict class of first test image
pred = model.predict(X_test[:1])          # Get probability array
print("Predicted:", class_names[np.argmax(pred)]) # Pick highest prob class
print("Actual:",    class_names[y_test[0]])
```